# Figure catalogue

Every figure in `docs/report_figures/`, with what it asks, where the data comes from, how to read it and what it shows. Folders follow the order the report presents things.

Generated by `make_report_figures.py` from the same list that writes the figures, so the two cannot disagree. Each figure also exists as a 300 dpi PNG beside the SVG.

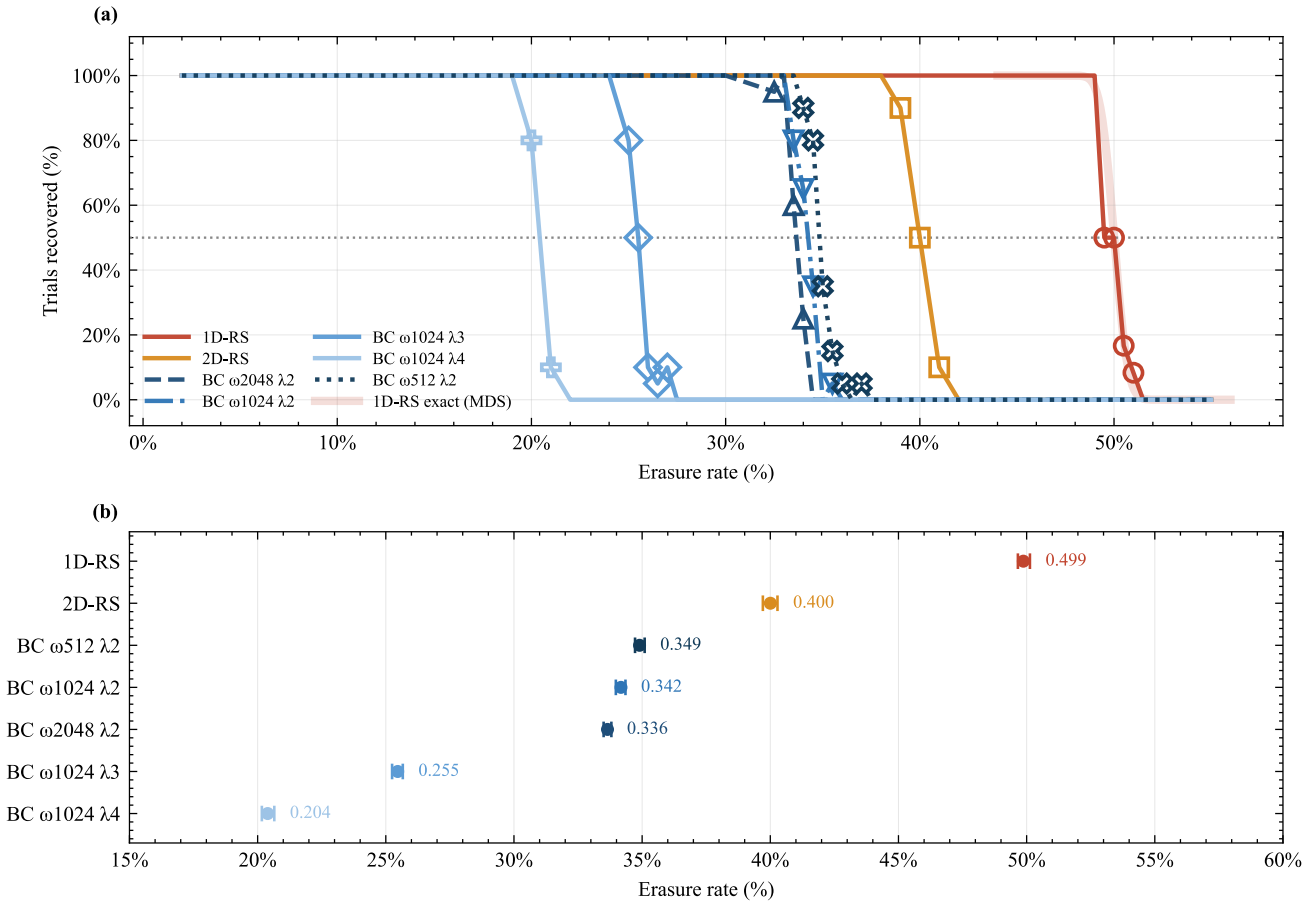
**18 figures** — 9 in `01-benchmark`, 8 in `02-network-sim`, 1 in `03-methodology`.

---

## The coding benchmark

*`01-benchmark/` — Seven schemes over the same blob, same erasure positions, at matched storage cost. This is the first results chapter.*

### 01. fault tolerance




**Goes in:** Results — fault tolerance

**The question.** How much of the codeword can each scheme lose and still rebuild the blob exactly?

**Where the data comes from.** The erasure sweep: one run over one real EIP-4844 blob, covering 2% to 55% erasure, with dense trials on a fine grid across each scheme's own transition band (benchmark_results.json).

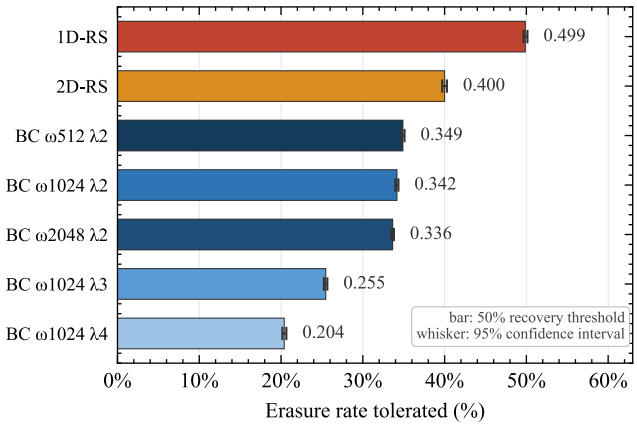
**How to read it.** (a) Fraction of trials recovered against erasure probability, one line per scheme; the pale band is the exact recovery probability of 1D-RS, computed from the MDS property rather than fitted. (b) Each scheme's threshold -- the erasure probability at which it recovers half the time -- with its 95% bootstrap confidence interval, from a maximum-likelihood logistic fit.

**What it shows.** Every scheme holds at 100% then collapses within a few percent, which is what makes a single threshold a fair summary. 1D-RS sits on its exact MDS curve, at 0.499. The three lambda=2 rows land within 0.013 of each other (0.336, 0.342, 0.349), far closer to one another than to lambda=3 (0.255) or lambda=4 (0.204): lambda sets the reach. Their intervals do not overlap, though, and they rise with mu - so mu has a small, real effect.

**Worth noting.** Erasures are drawn i.i.d., so these are upper bounds on performance against an adversary choosing the worst pattern.

`01-benchmark/01-fault-tolerance.svg` and `.png` (300 dpi)

### 02. fault tolerance compact




**Goes in:** Results — fault tolerance (space-saving alternative to 01)

**The question.** Where does each scheme fall over, and how close together are those points?

**Where the data comes from.** The erasure sweep's dense trials across each scheme's transition band, 12-20 per rate (benchmark_results.json).

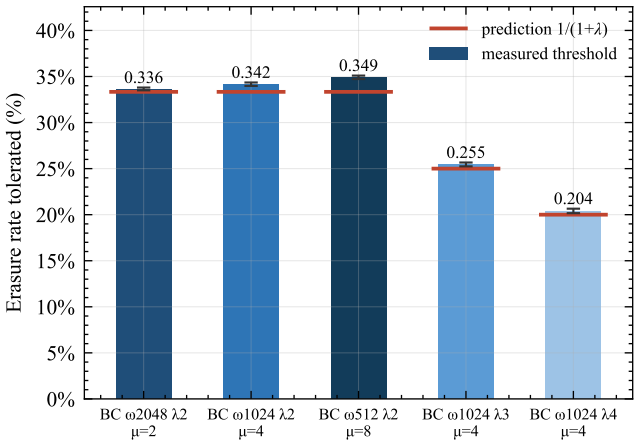
**How to read it.** Horizontal bars, one per scheme, sorted by threshold. Bar length is the erasure probability at which the scheme recovers half the time, from a maximum-likelihood logistic fit; the whisker is its 95% bootstrap confidence interval.

**What it shows.** 1D-RS reaches 0.499, against the exact 0.50006 an MDS code with n = 2k can sustain. The three lambda=2 rows sit within 0.013 of each other (0.336, 0.342, 0.349) while lambda=3 and lambda=4 fall to 0.255 and 0.204. Intervals are +/-0.003 or narrower, so every difference shown is resolved.

**Worth noting.** A standalone, single-column alternative to figure 01. Use one or the other, not both: they are the same data. This one carries the comparison and is readable without its caption; figure 01 adds the shape of each transition, which matters only if the prose leans on it.

`01-benchmark/02-fault-tolerance-compact.svg` and `.png` (300 dpi)

## 03. bc threshold vs theory




**Goes in:** Results — fault tolerance

**The question.** Does the block-circulant decoder behave as the source paper predicts?

**Where the data comes from.** Thresholds from the erasure sweep, with 95% intervals; the prediction is the local code's stopping rate from Sasidharan, Viterbo & Feist (ePrint 2026/708).

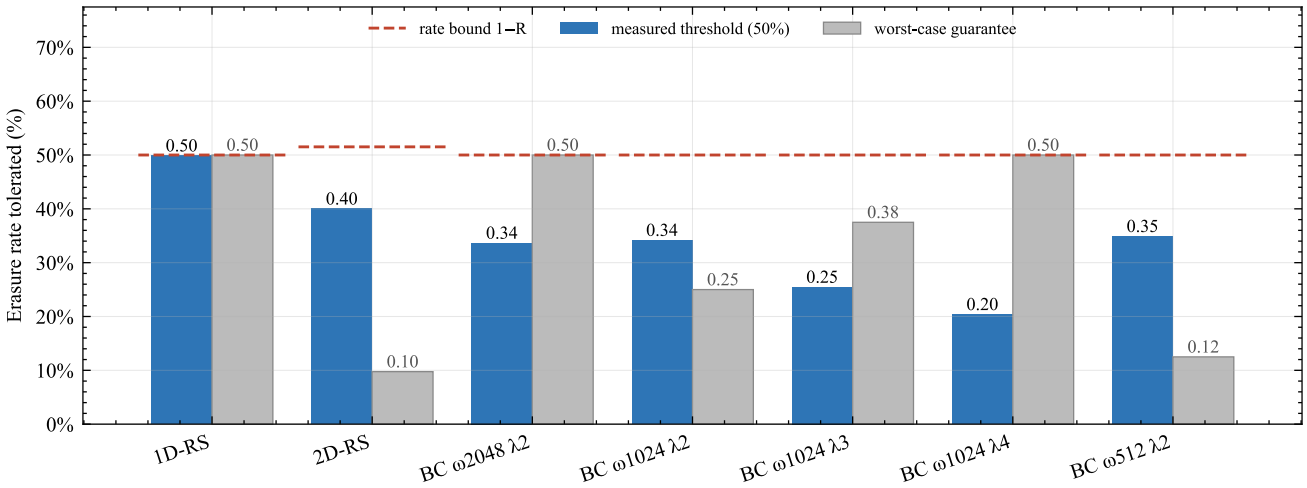
**How to read it.** Bars are measured thresholds with 95% confidence intervals, ordered by lambda and then by mu. Red rules are the predicted local stopping rate S0 = rho/n0 = 1/(1+lambda).

**What it shows.** Lambda sets the reach. Predicted 0.333/0.250/0.200 for lambda = 2/3/4 against measured 0.336-0.349, 0.255 and 0.204. Every measurement sits just above its prediction, as it should: the local stopping rate is what one block can repair alone, and the cascade across blocks can only add to it. At fixed lambda=2 the threshold also rises with mu (0.336, 0.342, 0.349 for mu = 2, 4, 8) - more blocks give that cascade more to work with.

**Worth noting.** An earlier estimate put lambda=3 at 0.275. It was the midpoint of a gap between two coarse grid points with no trial inside it; dense sampling across the band places it just above the prediction, at 0.255.

`01-benchmark/03-bc-threshold-vs-theory.svg` and `.png` (300 dpi)

## 04. tolerance vs guarantee



**Goes in:** Discussion — guarantees versus behaviour

**The question.** How does what each code actually tolerated compare with what its minimum distance guarantees?

**Where the data comes from.** Same sweep for the measured bars; guarantees computed from each code's d_min, and the rate bound from 1 - R.

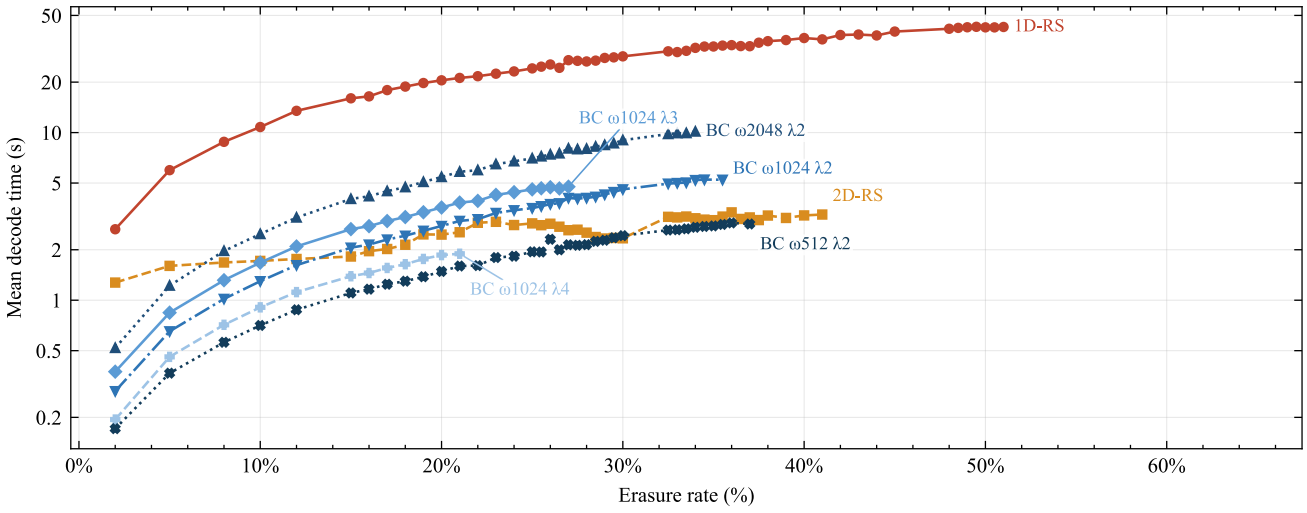
**How to read it.** Blue is the measured 50% threshold, grey the worst-case guarantee from minimum distance, dashed red the rate bound 1-R that no code of that rate can beat.

**What it shows.** 1D-RS sits on its bound at 0.499, which is what being MDS means and a useful check that the harness is sound. 2D-RS reaches 0.400 against a guarantee of 0.098 - four times its own bound.

**Worth noting.** Not a contradiction. A minimum-distance guarantee holds against every pattern including the adversarial worst case; these trials draw patterns at random.

`01-benchmark/04-tolerance-vs-guarantee.svg` and `.png` (300 dpi)

### 05. decode cost




**Goes in:** Results — computational cost

**The question.** What does each scheme cost to decode, and how does that grow with damage?

**Where the data comes from.** Same sweep. Mean over the trials that actually recovered; failed trials are excluded because a scheme above its threshold rejects the pattern on a budget check in under a millisecond, which is not a decode time.

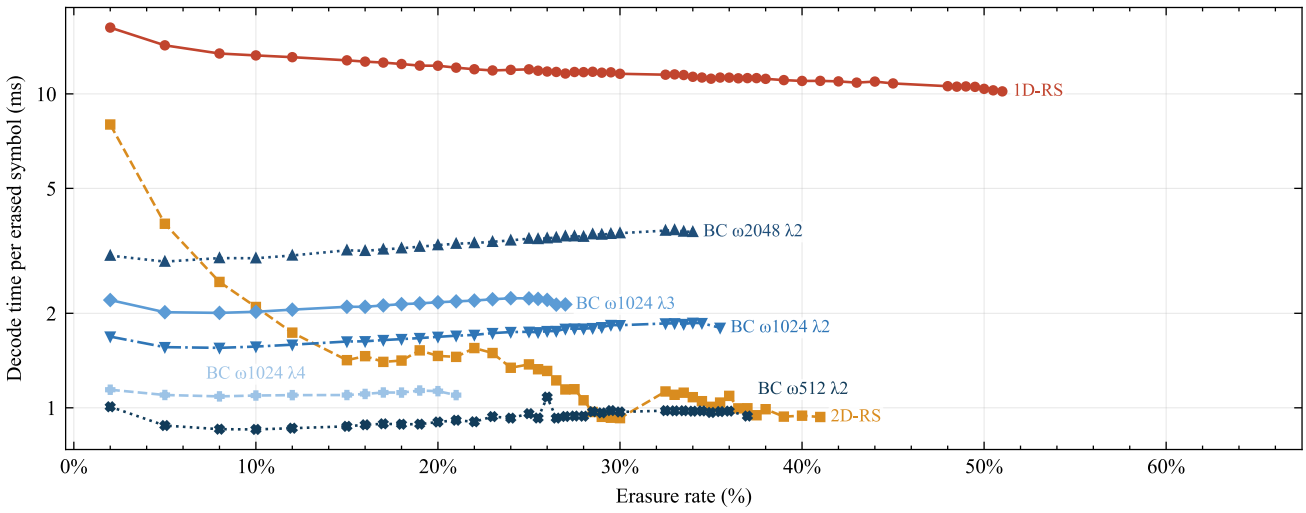
**How to read it.** Log vertical axis in seconds, erasure rate horizontal, curves named at their right-hand ends.

**What it shows.** 1D-RS is the most expensive at every rate, three to five times the dearest block-circulant row, because it divides one 8192-length polynomial while the others divide short ones. Against 2D-RS the picture is mixed: below about 7% every BC row is cheaper; above it they cross one by one, and only omega512 lambda2 and omega1024 lambda4 stay at or below it.

**Worth noting.** Single-threaded Python over galois. Relative comparisons hold; absolute seconds would not survive an optimised build.

`01-benchmark/05-decode-cost.svg` and `.png` (300 dpi)

### 06. cost per erasure




**Goes in:** Discussion — why the costs differ

**The question.** Is one scheme dearer because it does more work per repair, or because it carries a fixed overhead?

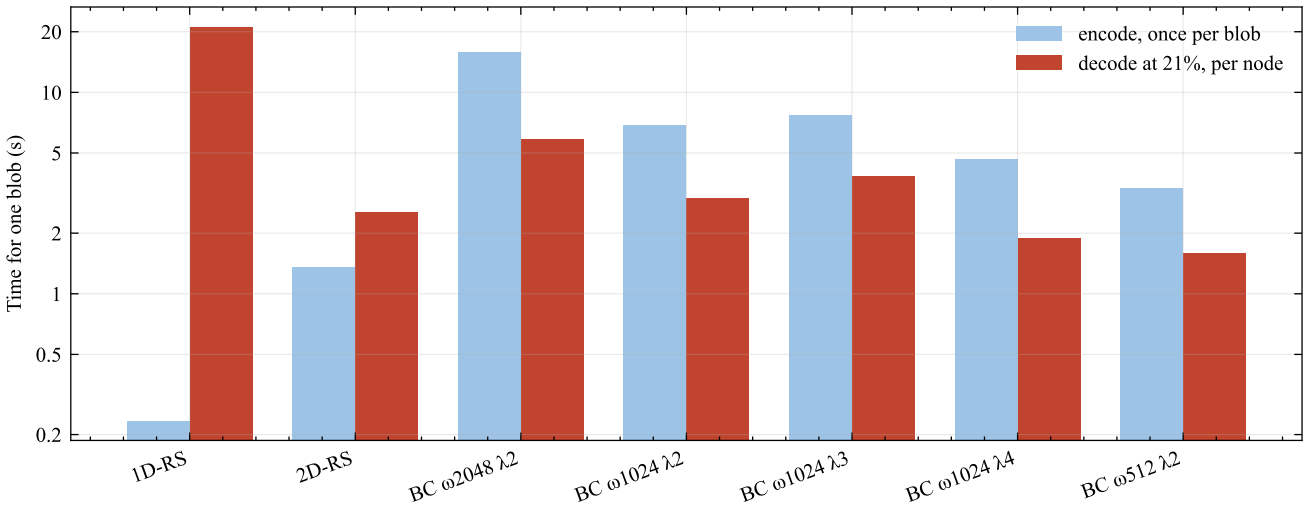
**Where the data comes from.** The same decode times divided by the number of symbols actually erased in that trial.

**How to read it.** Milliseconds per erased symbol, log axis. A flat line means cost is linear in the number of erasures; a falling line means a fixed cost is being amortised over more repairs.

**What it shows.** 1D-RS sits near 11-13 ms per erasure (16 ms at 2%, easing to 10.5 ms near its threshold) - one long division's worth of work each. The BC rows sit at 0.85-3.7 ms and rise gently. 2D-RS falls steeply from 8 ms to 1 ms, revealing a large fixed cost, which is exactly why it looks expensive at 2% erasure and cheap at 30%.

`01-benchmark/06-cost-per-erasure.svg` and `.png` (300 dpi)

### 07. encode vs decode



**Goes in:** Discussion — asymmetry

**The question.** Encoding and decoding are not paid by the same party. How do the schemes differ on each?

**Where the data comes from.** Encode time measured once per scheme on the shared blob; decode at the highest rate every scheme still recovers.

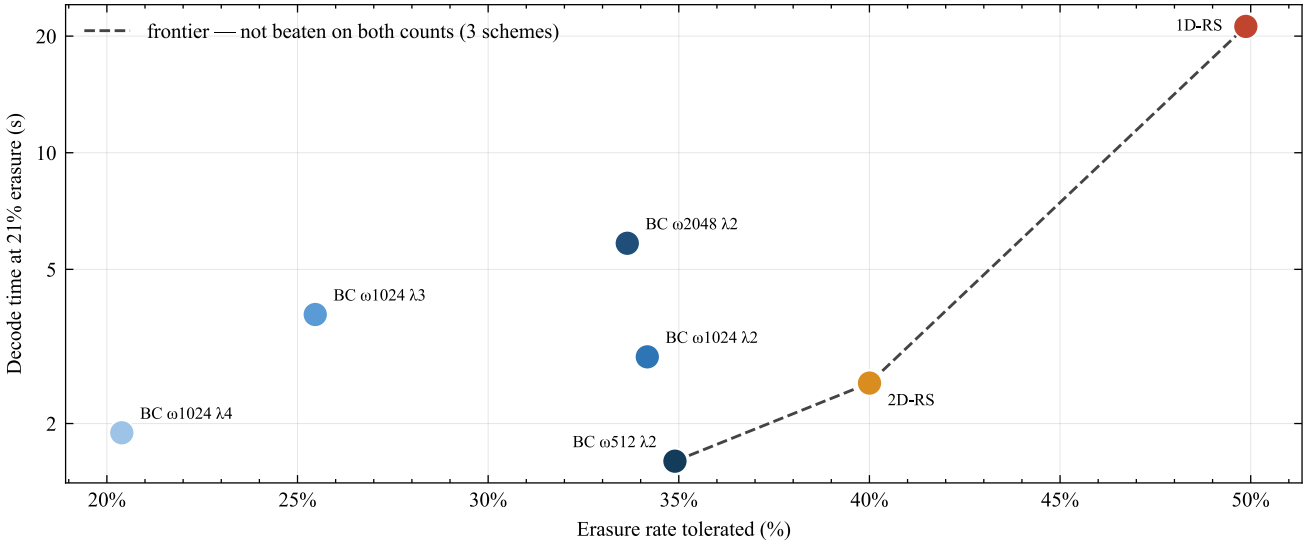
**How to read it.** Paired bars per scheme, log axis in seconds.

**What it shows.** The two families sit on opposite sides of the asymmetry. 1D-RS encodes in 0.23 s then costs 21 s to decode; BC omega2048 lambda2 spends 15.8 s encoding in order to decode in 5.8 s.

**Worth noting.** A blob is encoded once by the proposer and decoded by every node reconstructing it, so an expensive encode amortises across the network and an expensive decode does not. For DAS the second shape is the useful one - an argument for block-circulant the tolerance numbers alone do not make.

`01-benchmark/07-encode-vs-decode.svg` and `.png` (300 dpi)

### 08. pareto tolerance vs cost




**Goes in:** Discussion — the trade-off

**The question.** Which schemes are actually worth considering once tolerance and cost are weighed together?

**Where the data comes from.** Thresholds and decode times from the same sweep, at the highest rate every scheme reached.

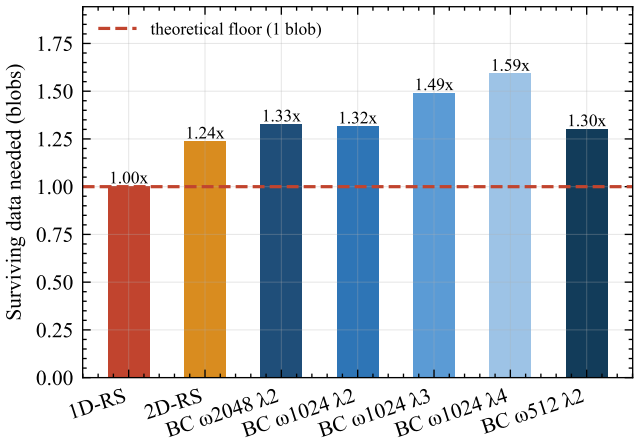
**How to read it.** One point per scheme: tolerance horizontal, decode cost vertical on a log axis. Down and to the right is better. The dashed line is the Pareto frontier - schemes not beaten on both counts at once.

**What it shows.** Only three of seven are on the frontier. BC omega512 lambda2 is the cheapest thing still surviving a third of the codeword; 2D-RS buys about five more points of tolerance for about 60% more decode time; 1D-RS buys the maximum and pays an order of magnitude. BC omega1024 lambda4 is dominated on both axes by omega512 lambda2, so the tolerance it gives up buys nothing.

**Worth noting.** If the report has one summary figure, this is it.

`01-benchmark/08-pareto-tolerance-vs-cost.svg` and `.png` (300 dpi)

### 09. data needed per blob




**Goes in:** Results — bandwidth

**The question.** How much surviving data must a node obtain to rebuild one blob?

**Where the data comes from.** Derived from each scheme's measured threshold as n(1-threshold)/k, expressed as a multiple of one blob.

**How to read it.** Bars per scheme against a dashed line at 1.0, the theoretical floor - nothing rebuilds 124 KiB from less than 124 KiB.

**What it shows.** 1D-RS needs essentially the floor, 1.00x. Everything else needs 1.24x to 1.59x. A scheme tolerating a lower erasure rate at the same storage cost necessarily needs more.

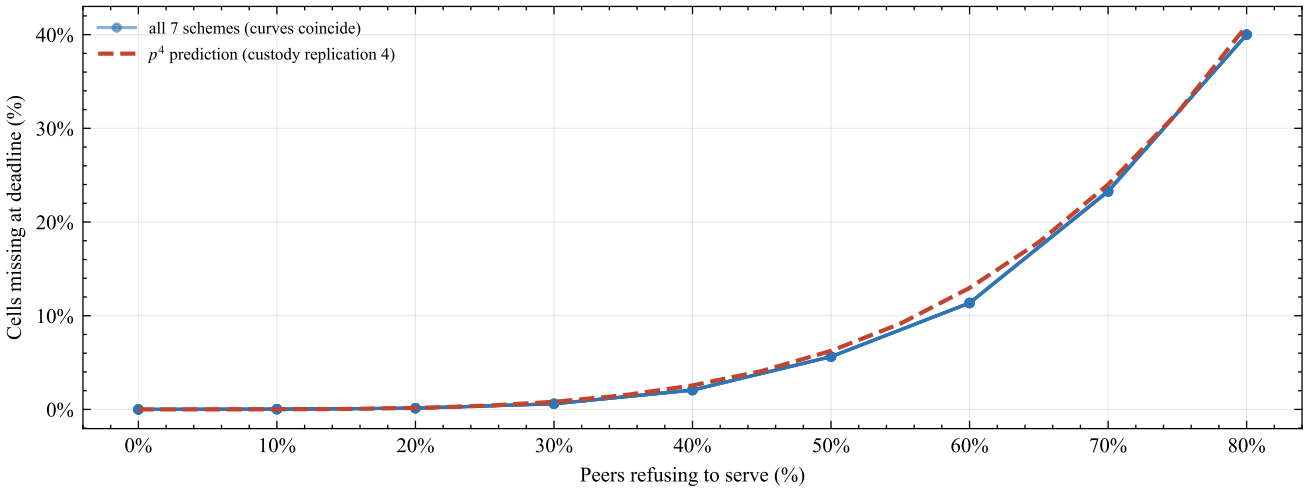
**Worth noting.** Computed from the measured thresholds. Network figure 04 shows what a node actually pulls; its download target is set from the same measured tolerance, so the shared ordering is by construction, and the network figure adds the real overheads on top.

`01-benchmark/09-data-needed-per-blob.svg` and `.png` (300 dpi)

## The network simulation

`02-network-sim/` — *The same schemes inside a simulated peer-to-peer network, where the erasure pattern is an output rather than an input. This follows the benchmark.*

## 01. erasure vs adversaries




**Goes in:** Results — network layer: fault tolerance

**The question.** When peers withhold data, what erasure rate does the code actually end up facing?

**Where the data comes from.** SimPy simulation, 64 peers, custody replication 4, one blob per slot at 50 Mbps, 4 s deadline, 30 trials per point, withholding swept 0 to 80%. docs/runA.json.

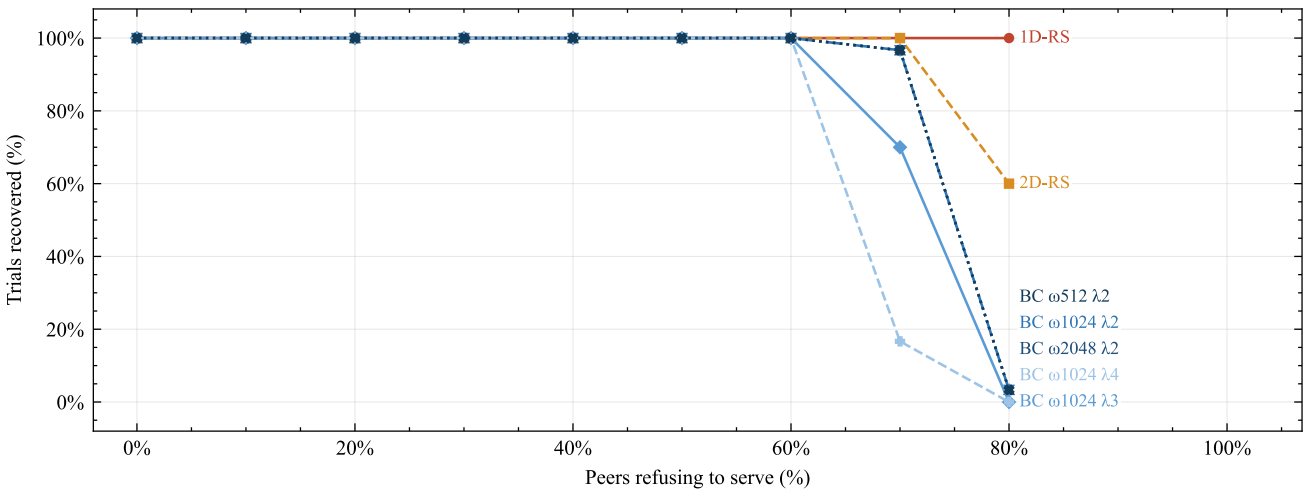
**How to read it.** Realised erasure against the fraction of peers refusing to serve, with the p^4 prediction dashed. All seven schemes are drawn; they coincide.

**What it shows.** A cell is lost only if all four of its custodians withhold, so the erasure the code sees is roughly the fourth power of the bad-peer fraction: 50% of the network withholding yields about 5.6% erasure. Every scheme gives the same curve - 2D-RS, with 132 cells rather than 128, differs only in the fourth decimal - because this is a property of the network and custody, not the code.

**Worth noting.** Deflating for the coding schemes: replication absorbs most peer failure before the erasure code is asked to do anything.

`02-network-sim/01-erasure-vs-adversaries.svg` and `.png` (300 dpi)

## 02. recovery vs adversaries



**Goes in:** Results — network layer: fault tolerance

**The question.** At what level of peer withholding does each scheme stop recovering the blob?

**Where the data comes from.** Same run. No erasure rate is injected here - losses are whatever failed to arrive before the 4 s deadline. Whether a realised erasure is survivable is read from the coding benchmark's measured recovery curve (calibrated mode).

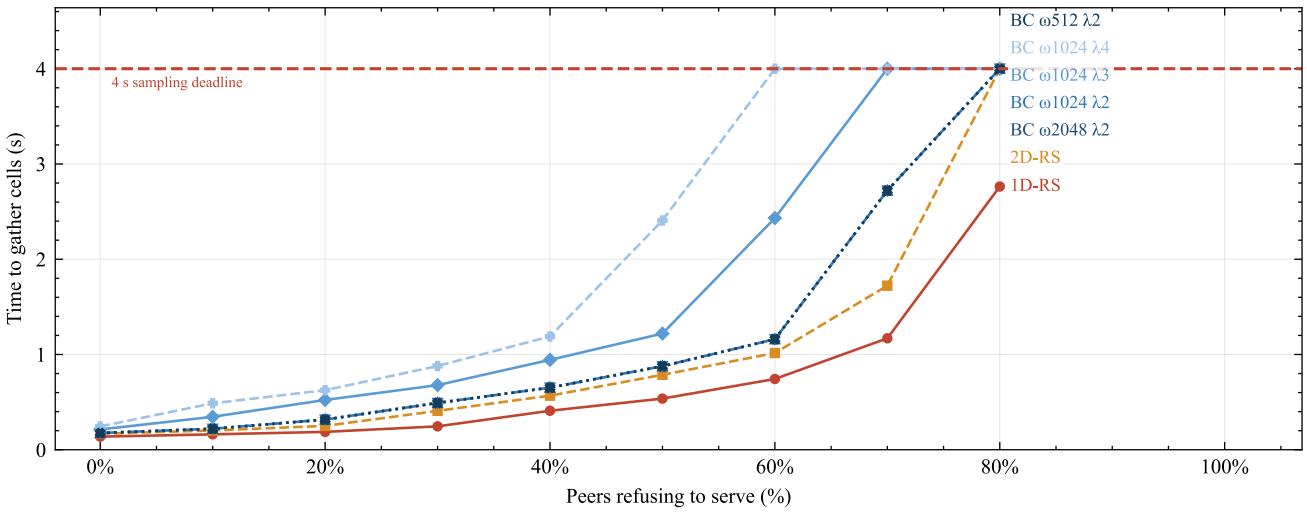
**How to read it.** Fraction of 30 trials that rebuilt the blob byte-exactly, against withholding.

**What it shows.** BC omega1024 lambda4 breaks first (5/30 at 70% withholding, which yields 0.233 erasure against its 0.204 threshold), with omega1024 lambda3 close behind (21/30). 2D-RS and the lambda2 rows hold until 80%, where erasure reaches 0.400; 1D-RS is still at 30/30 when 80% of peers refuse.

**Worth noting.** The failure order is inherited from the benchmark by construction - recovery is calibrated, not re-decoded - so this translates the benchmark's thresholds into withholding terms rather than confirming them independently. Running the same points with --real-decode would make it a confirmation.

`02-network-sim/02-recovery-vs-adversaries.svg` and `.png` (300 dpi)

### 03. latency gather time




**Goes in:** Results — network layer: latency

**The question.** Does withholding cost the victim anything beyond the data itself?

**Where the data comes from.** Same run. Gather time is network time only, excluding decode.

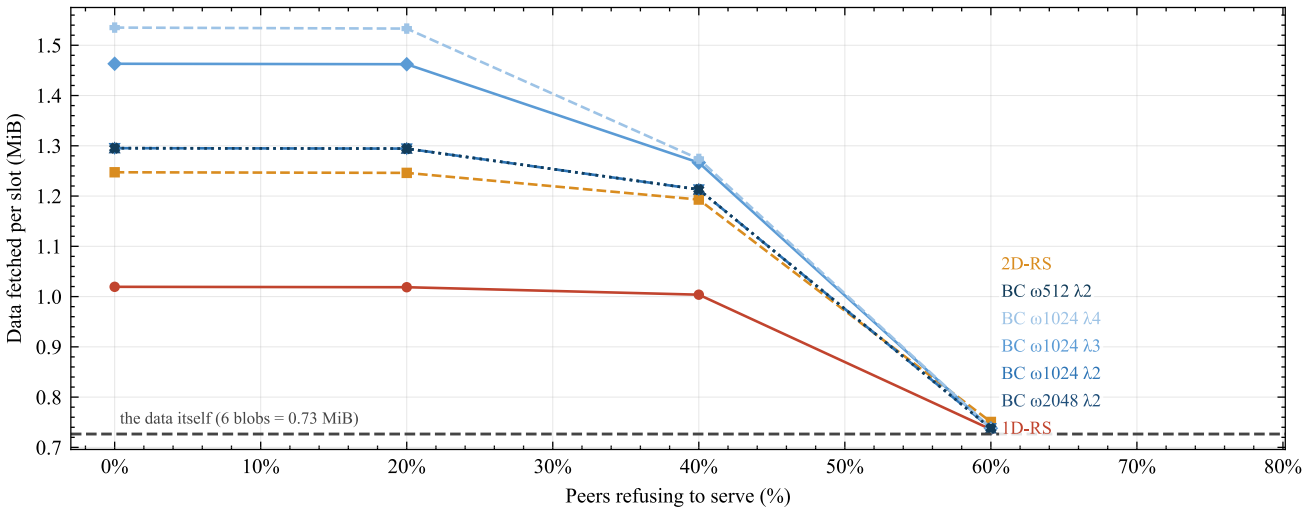
**How to read it.** Seconds to collect the cells, against withholding, with the 4 s sampling deadline marked. A curve reaching 4.00 has run out of time.

**What it shows.** Yes - it costs time. A peer that never answers cannot be routed around for free: the client burns its request timeout before falling back to another custodian. Gather climbs from 0.14 s to 2.76 s for 1D-RS across the sweep. The scheme needing the most cells hits the deadline first - omega1024 lambda4, at 60% withholding - while 1D-RS never reaches it.

**Worth noting.** Covers the brief's 'packet propagation latency'. Invisible in any figure measured only at zero adversaries.

`02-network-sim/03-latency-gather-time.svg` and `.png` (300 dpi)

### 04. bandwidth fetched




**Goes in:** Results — network layer: bandwidth

**The question.** How much data does a node actually pull over the wire, as opposed to how much the code theoretically needs?

**Where the data comes from.** SimPy, 64 peers, replication 4, six blobs per slot at 25 Mbps - the bandwidth-pressure configuration. 30 trials per point. docs/runB.json.

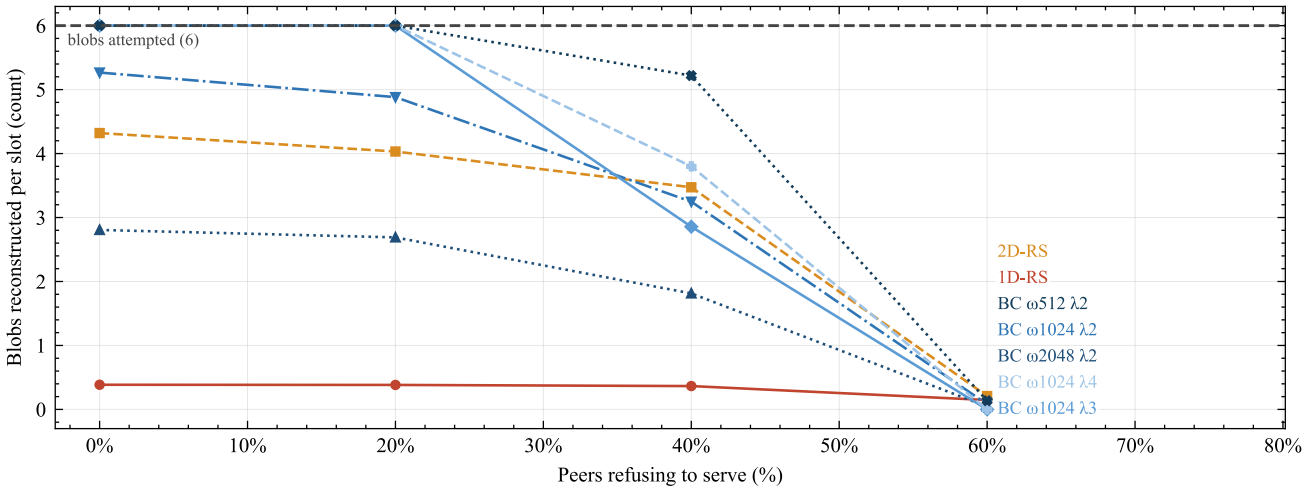
**How to read it.** MiB fetched per slot against withholding, with a dashed line at the actual payload (six blobs = 0.73 MiB). Includes the over-request margin and anything re-requested after a timeout.

**What it shows.** 1D-RS pulls least at 1.02 MiB and BC omega1024 lambda4 most at 1.54 MiB, the whole codeword. The ordering matches benchmark figure 09 by construction - the download target is set from the same measured tolerance - and what this figure adds is the real overhead on top: the over-request margin, requests already in flight, and re-requests after timeouts.

**Worth noting.** Read the fall at high withholding carefully: that is cells that never arrived, not a saving.

`02-network-sim/04-bandwidth-fetched.svg` and `.png` (300 dpi)

## 05. throughput blobs per slot




**Goes in:** Results — network layer: throughput

**The question.** How many blobs can a node actually rebuild within one slot?

**Where the data comes from.** Same six-blob run. Throughput implied by the measured time - blobs attempted, scaled by recovery rate and by the share of the work that fits in the 12 s slot - not an independently measured rate.

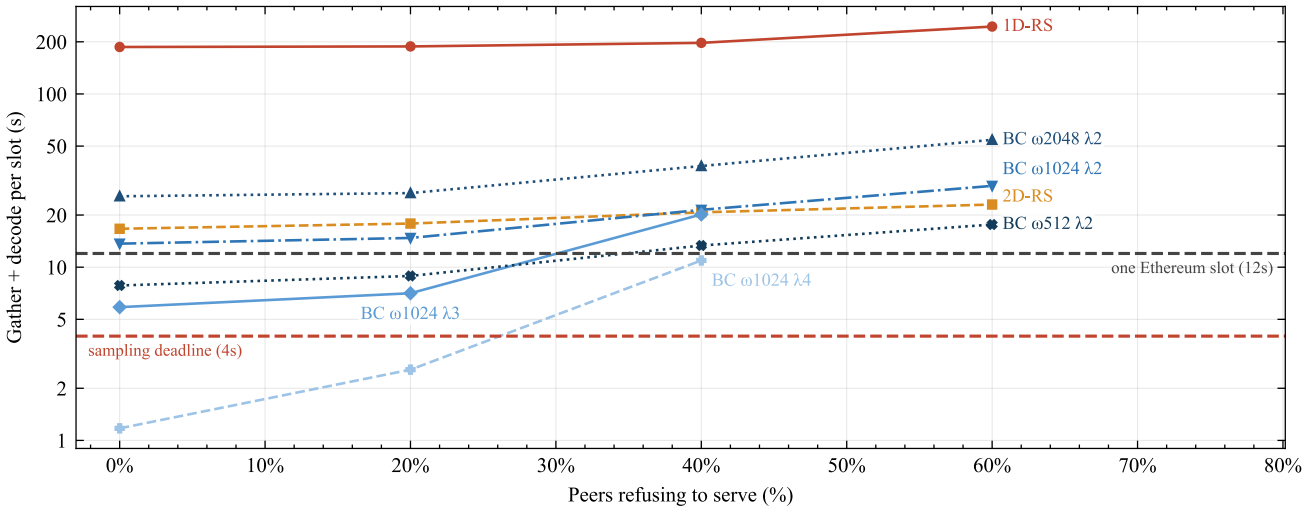
**How to read it.** Blobs reconstructed per slot out of six, against withholding, with a dashed line at six.

**What it shows.** The ordering broadly inverts the tolerance ordering. 1D-RS tolerates the most erasure of any scheme and reconstructs the fewest blobs, about 0.4 of the six, because one 8192-length polynomial division costs about 31 s per blob. BC omega1024 lambda3 and lambda4 keep up with all six until 20% withholding, and so does omega512 lambda2; at zero withholding omega1024 lambda2 manages 5.3, 2D-RS 4.3 and omega2048 lambda2 2.8. Two rows break a clean inversion: omega2048 lambda2, whose large blocks are slow to decode, and omega512 lambda2, which keeps up with all six while being the most tolerant block-circulant row.

**Worth noting.** Covers the brief's 'block processing throughput'.

`02-network-sim/05-throughput-blobs-per-slot.svg` and `.png` (300 dpi)

## 06. time budget vs slot




**Goes in:** Discussion — can any of this keep up?

**The question.** Does the whole job - fetch plus rebuild - fit inside an Ethereum slot?

**Where the data comes from.** Same six-blob run. Gather plus decode summed per slot.

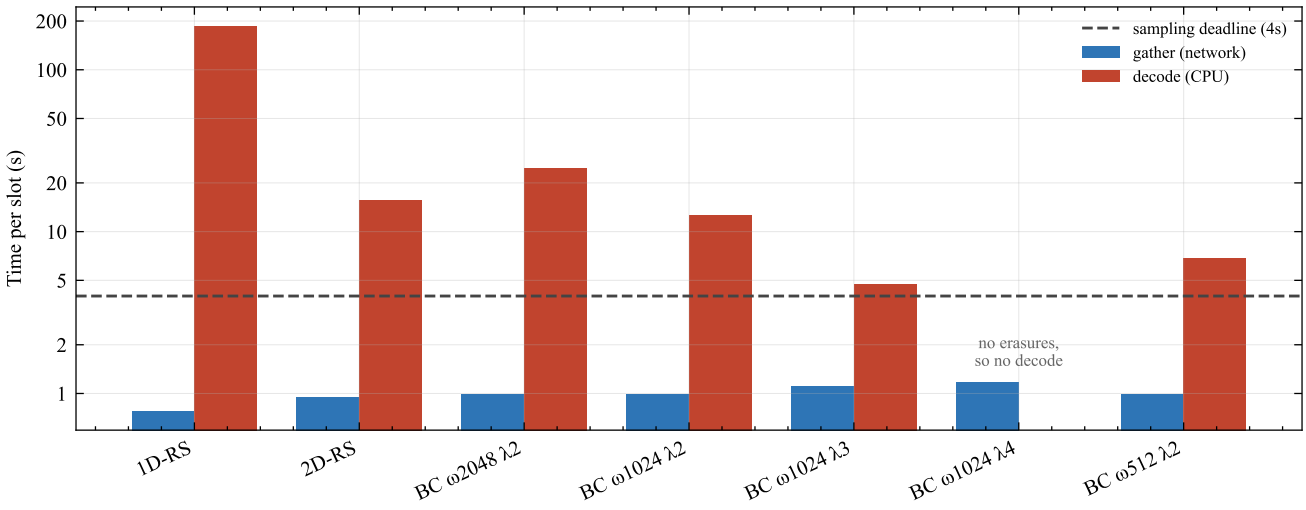
**How to read it.** Log seconds against withholding, with the 4 s sampling deadline and the 12 s slot both marked. Anything above the 12 s line cannot keep up with the chain.

**What it shows.** At zero adversaries only BC omega512 lambda2 (7.8 s), omega1024 lambda3 (5.9 s) and lambda4 (1.2 s) finish inside a slot, with omega1024 lambda2 just over at 13.7 s. By 40% withholding only lambda4 still averages under 12 s, and it recovers in only 19 of 30 trials. 1D-RS is not close at any setting - 187 s, roughly fifteen slots of work for one slot of data.

**Worth noting.** The figure that reframes the comparison: erasure tolerance is not the binding constraint, decode throughput is.

`02-network-sim/06-time-budget-vs-slot.svg` and `.png` (300 dpi)

## 07. gather vs decode




**Goes in:** Discussion — the real bottleneck

**The question.** Of the time a node spends, how much is network and how much is arithmetic?

**Where the data comes from.** Same six-blob run, at zero adversaries so neither side is inflated by timeouts.

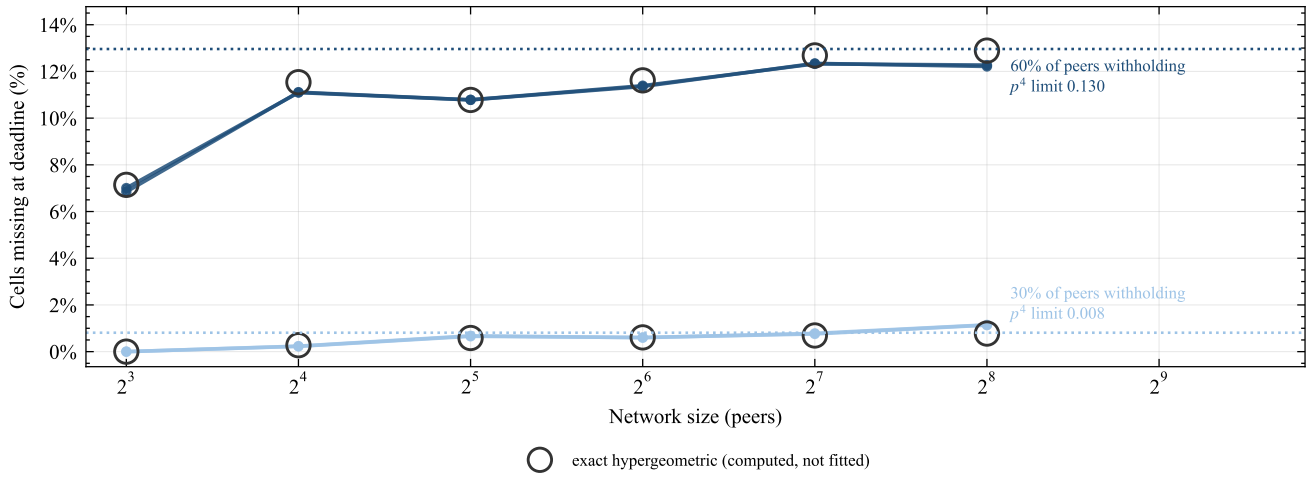
**How to read it.** Gather and decode as separate bars per scheme on a log axis - separate rather than stacked, because stacking on a log axis would misrepresent the sum. The 4 s sampling deadline is marked.

**What it shows.** Every scheme gathers in about one second. Decode runs from effectively zero - omega1024 lambda4, which ends up holding the whole codeword - to 186 s for 1D-RS. Apart from that one scheme, arithmetic rather than bandwidth or latency decides the outcome.

**Worth noting.** Bandwidth was the constraint this project expected to find binding. It is not close.

`02-network-sim/07-gather-vs-decode.svg` and `.png` (300 dpi)

### 08. scalability peer count




**Goes in:** Results — network layer: scalability

**The question.** What happens to data availability as the network grows?

**Where the data comes from.** SimPy peer sweep, 8 to 256 peers, replication fixed at 4, one blob at 50 Mbps, 30 trials per point, at 30% and 60% withholding. docs/runC.json.

**How to read it.** Realised erasure against peer count on a log-2 axis. Open markers are the exact hypergeometric probability that all four custodians of a cell are adversarial, C(A,4)/C(n,4), computed from the model rather than fitted. Dotted lines are the p^4 independent-sampling limits.

**What it shows.** Growing the network makes things slightly worse, up to a limit. Custodians are drawn without replacement, so in a small network it is combinatorially hard for all four to be adversarial - with 8 peers only 5 are bad. As the network grows that constraint relaxes and loss climbs toward p^4 = 0.130. Measurement sits on the computed prediction throughout.

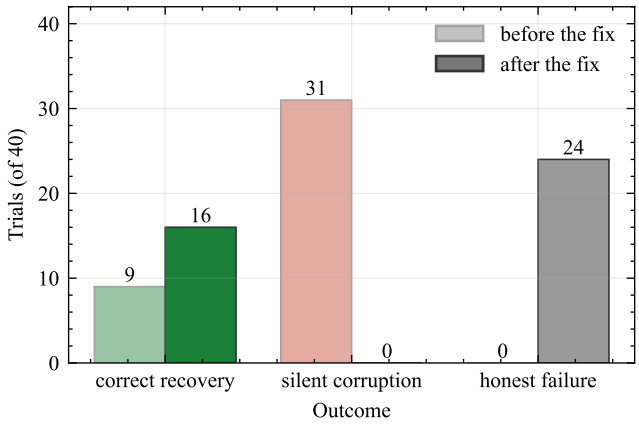
**Worth noting.** The lesson for scalability claims: what protects a cell is the fraction of the network custodying it, so peer count cannot be varied without saying what replication does too.

`02-network-sim/08-scalability-peer-count.svg` and `.png` (300 dpi)

## Methodology and validation

`03-methodology/` — *Referenced from the method chapter rather than the results.*

### 01. decoder correctness fix




**Goes in:** Methodology — validation

**The question.** Why does this project verify recovered bytes rather than trusting the decoder's own success flag?

**Where the data comes from.** A historical run at omega=rho=lambda=2, mu=2048 - a parameterisation the main sweep does not cover - with the same 40 trials (ten each at 5%, 15%, 25% and 45% erasure) put through the decoder before and after a correctness fix. decoder_fix_results.json.

**How to read it.** Outcome counts for the same 40 trials, before the fix in pale and after in solid. Three outcomes: correct recovery, silent corruption, honest failure.

**What it shows.** Before the fix the decoder claimed success on all 40 trials and was wrong on 31 of them; it never once reported a failure. After, silent corruption is zero and correct recoveries rose from 9 to 16 - the old decoder had been poisoning positions with fabricated zeros that blocked later rounds from resolving them.

**Worth noting.** Had the harness trusted the decoder's 'remaining' flag it would have reported 100% success at every erasure rate. The strongest validation argument the project has.

`03-methodology/01-decoder-correctness-fix.svg` and `.png` (300 dpi)